

C Fundamentals for Embedded Systems

A Ground-Up Reference — EE0827 Embedded System Applications

How to Use This Sheet

This document starts from **zero assumptions** about your C knowledge and builds up to the level needed for this course. Every concept includes a memory diagram, a worked example, and common mistakes. Work through it with a compiler open — type the examples, predict the output, then verify.

1. Variables, Types, and Memory

What is a variable?

A variable is a **named region of memory** that holds a value. When you write `int x = 42;`, the compiler:

1. Reserves space in memory (4 bytes for `int` on ARM Cortex-M)
2. Associates the name `x` with that memory address
3. Writes the value 42 into those bytes

```
int x = 42;
```

Memory (RAM):

```
Address:  0x20000100  0x20000101  0x20000102  0x20000103
Content:   [ 0x2A   ] [ 0x00   ] [ 0x00   ] [ 0x00   ]
           x = 42 (little-endian)
```

Little-endian: ARM stores the least-significant byte at the lowest address. $42 = 0x0000002A$, so `0x2A` comes first.

Fixed-width integer types (use these, not `int`)

In embedded C, you must know exactly how many bytes each variable occupies. The standard `int` can be 2 or 4 bytes depending on the platform. Use `<stdint.h>` types instead:

Type	Size	Range	Use case
<code>uint8_t</code>	1 byte	0 to 255	Byte buffers, register bytes
<code>int8_t</code>	1 byte	-128 to 127	Signed small values
<code>uint16_t</code>	2 bytes	0 to 65,535	ADC values (12-bit), PWM
<code>int16_t</code>	2 bytes	-32,768 to 32,767	Temperature with sign
<code>uint32_t</code>	4 bytes	0 to 4,294,967,295	Timestamps, registers
<code>int32_t</code>	4 bytes	± 2.1 billion	General-purpose signed

```
#include <stdint.h>
```

```
uint8_t sensor_id = 3; // 1 byte
```

```
uint16_t adc_reading = 2048;    // 2 bytes
uint32_t timestamp  = 0;       // 4 bytes - can count to 4 billion
```

Common Mistake: Integer Overflow

```
uint8_t counter = 255; counter++; // counter is now 0, not 256!
Unsigned integers wrap around silently. If you need values above 255, use uint16_t or uint32_t.
```

Type casting and implicit conversions

C will silently convert between types, sometimes losing data:

```
uint16_t big = 300;
uint8_t  small = big;    // small = 44 (300 - 256), data LOST silently!

int8_t  signed_val = -1;
uint8_t unsigned_val = signed_val; // unsigned_val = 255 (reinterpreted)
```

Rule: Never assign a wider type to a narrower type without checking the range first. The compiler may warn, but it will still compile.

Hexadecimal notation

Hex is essential for embedded work because each hex digit maps to exactly 4 bits:

Hex	Binary	Decimal
0x0	0000	0
0x1	0001	1
0x5	0101	5
0x9	1001	9
0xA	1010	10
0xF	1111	15
0xFF	1111 1111	255
0xDEAD	1101 1110 1010 1101	57005

Converting: $0x2A = 0010\ 1010 = 2 \times 16 + 10 = 42$

```
uint32_t reg = 0x40020014; // A peripheral register address
uint8_t  mask = 0xFF;      // All 8 bits set
uint32_t flag = 0x00000020; // Only bit 5 set (1 << 5)
```

Why Hex?

Register addresses and bitmasks are always written in hex because you can instantly see which bits are set. `0x20` tells you “bit 5” at a glance. `32` in decimal does not.

2. Operators

Arithmetic operators

```
int a = 17, b = 5;

a + b    // 22      addition
a - b    // 12      subtraction
a * b    // 85      multiplication
a / b    // 3        integer division (truncates toward zero)
a % b    // 2        modulo (remainder): 17 = 3*5 + 2
```

Integer division trap: 7 / 2 is 3, not 3.5. Both operands are integers, so the result is an integer.

Compound assignment

```
x += 5;    // same as x = x + 5
x -= 3;    // same as x = x - 3
x *= 2;    // same as x = x * 2
x /= 4;    // same as x = x / 4
x |= mask; // same as x = x | mask    (used heavily for registers)
x &= mask;  // same as x = x & mask
x ^= mask;  // same as x = x ^ mask
```

Increment and decrement

```
int x = 5;

x++;    // post-increment: use x, then add 1 → x is now 6
++x;    // pre-increment: add 1, then use x → x is now 7
x--;    // post-decrement: use x, then subtract 1 → x is now 6
```

In embedded code: prefer `x++` or `++x` as standalone statements. Avoid using them inside expressions like `arr[i++]` until you're comfortable — the evaluation order is a common source of bugs.

Comparison and logical operators

```
// Comparison (return 1 for true, 0 for false)
a == b    // equal to          (TWO equals signs!)
a != b    // not equal to
a < b     // less than
a <= b    // less than or equal
a > b     // greater than
a >= b    // greater than or equal

// Logical (combine conditions)
a && b     // logical AND: true if BOTH are true
a || b     // logical OR: true if EITHER is true
!a         // logical NOT: true becomes false, false becomes true
```

`if (x = 5)` assigns 5 to x (always true). You meant `if (x == 5)`.
Some programmers write `if (5 == x)` so that `if (5 = x)` won't compile. This is called a "Yoda condition."

3. Control Flow

if / else if / else

```
uint16_t adc = read_adc();

if (adc > 3000) {
    status = OVERVOLTAGE;
} else if (adc > 2000) {
    status = NORMAL;
} else {
    status = UNDERVOLTAGE;
}
```

Braces rule: Always use braces {}, even for single-line bodies. Omitting them is a well-known source of bugs.

for loop

```
// Print numbers 0 through 9
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}

// Initialize a buffer to zero
uint8_t buf[64];
for (int i = 0; i < 64; i++) {
    buf[i] = 0;
}
```

Structure: `for (init; condition; update)` — init runs once, condition is checked before each iteration, update runs after each iteration.

while and do-while

```
// while - checks condition BEFORE body (may never execute)
while (USART2->SR & (1 << 5) == 0) {
    // Wait for RXNE flag
}
uint8_t data = USART2->DR;

// do-while - checks condition AFTER body (always executes at least once)
do {
    data = read_sensor();
} while (data == 0xFF); // Retry until valid reading
```

switch

```
switch (command) {
    case 'L':
        led_on();
        break;           // MUST break, or falls through to next case!
    case 'O':
        led_off();
        break;
    default:
        printf("Unknown command: %c\r\n", command);
        break;
}
```

Forgetting break

Without **break**, execution **falls through** to the next case. This is legal C but almost always a bug. Every **case** should end with **break** unless you intentionally want fall-through (and add a comment if you do).

4. Functions

Anatomy of a function

```
// return_type  name      parameters
uint32_t      square    (uint32_t x)
{
    return x * x;        // returns a value to the caller
}

// void = returns nothing
void led_toggle(void) {    // void parameter = takes nothing
    GPIOA->ODR ^= (1 << 5);
}
```

Function declaration vs definition

```
// Declaration (prototype) - tells the compiler "this function exists"
// Usually placed in a header file (.h) or at the top of the .c file
uint32_t square(uint32_t x);

// Definition - the actual implementation
// Usually in the .c file
uint32_t square(uint32_t x) {
    return x * x;
}
```

Why separate? C compiles top-to-bottom. If `main()` calls `square()` but `square()` is defined below `main()`, the compiler hasn't seen it yet. The declaration (prototype) solves this.

Pass by value (C's default)

```
void double_it(int x) {
    x = x * 2;    // Modifies LOCAL copy only
}

int main(void) {
    int val = 5;
    double_it(val);
    // val is STILL 5 - the function got a copy
}
```

C always copies the argument value into the function's parameter. The original variable is untouched. To modify the original, you need **pointers** (covered next).

Pass by pointer (simulating pass-by-reference)

```
void double_it(int *x) {
    *x = (*x) * 2;    // Dereference pointer → modifies the ORIGINAL
}

int main(void) {
    int val = 5;
    double_it(&val);    // Pass ADDRESS of val
    // val is now 10
}
```

This is why HAL functions take pointers:

```
HAL_UART_Transmit(&huart2, data, len, timeout);
//                ~~~~~
//                address of the handle struct
```

5. Pointers — From the Ground Up

What is a pointer?

A pointer is a variable that **stores a memory address**. That's it.

```
int x    = 42;    // x is an int, holds value 42
int *ptr = &x;    // ptr is a pointer-to-int, holds address of x
```

Variable:	x	ptr
Value:	42	0x20000100
Address:	0x20000100	0x20000200

ptr "points to" x

The two pointer operators

Operator	Name	Meaning	Example
&	Address-of	“Give me the address of this variable”	<code>&x</code> → 0x20000100
*	Dereference	“Give me the value at this address”	<code>*ptr</code> → 42

```
int x = 42;
int *ptr = &x;    // ptr = address of x

printf("%d\n", x);    // 42          (value of x)
printf("%p\n", ptr);  // 0x20000100 (address stored in ptr)
printf("%d\n", *ptr); // 42          (value AT the address)

*ptr = 99;           // Write through pointer
printf("%d\n", x);   // 99          (x changed!)
```

Reading Pointer Declarations

Read **right to left**: `int *ptr` → “ptr is a pointer (*) to int.”
`volatile uint32_t *reg` → “reg is a pointer to a volatile `uint32_t`.”
The * in the declaration means “this is a pointer.” The * in usage means “dereference.”

Pointer arithmetic

When you add to a pointer, it advances by **the size of the pointed-to type**, not by bytes:

```
uint32_t arr[4] = {10, 20, 30, 40};
uint32_t *p = arr;    // p points to arr[0]

// p+0 → address of arr[0] (p + 0*4 bytes)
// p+1 → address of arr[1] (p + 1*4 bytes)
// p+2 → address of arr[2] (p + 2*4 bytes)
```

Address:	0x100	0x104	0x108	0x10C
Value:	[10]	[20]	[30]	[40]
	p+0	p+1	p+2	p+3
	*p	*(p+1)	*(p+2)	*(p+3)
	p[0]	p[1]	p[2]	p[3]

Key equivalence: `p[i]` is exactly the same as `*(p + i)`. Array indexing IS pointer arithmetic.

Pointer type	ptr + 1 advances by
<code>uint8_t *</code>	1 byte
<code>uint16_t *</code>	2 bytes
<code>uint32_t *</code>	4 bytes

Arrays decay to pointers

When you pass an array to a function, it **decays** to a pointer to its first element:

```

void print_array(uint32_t *arr, int len) {
    // arr is a pointer here, NOT an array
    // sizeof(arr) = 4 (pointer size), NOT the array size!
    for (int i = 0; i < len; i++) {
        printf("%u ", arr[i]);
    }
}

uint32_t data[100];
print_array(data, 100); // data decays to &data[0]

```

The sizeof Trap

Inside a function, `sizeof(arr)` gives the **pointer size** (4 bytes on Cortex-M), not the array size. **Always pass the length as a separate parameter.**
`sizeof` only gives the true array size when used on the original array declaration, not on a pointer.

NULL pointers

A pointer that doesn't point to anything should be set to NULL:

```

int *ptr = NULL; // ptr points to nothing

if (ptr != NULL) {
    *ptr = 42; // Safe - only dereference if not NULL
}

*ptr = 42; // BUG: dereferencing NULL → HardFault on Cortex-M

```

On ARM Cortex-M, dereferencing NULL (address 0x00000000) causes a **HardFault** exception and the MCU resets or halts. This is one of the most common firmware crashes.

Worked example: Trace through pointer code

```
int a = 10, b = 20;
int *p1 = &a;
int *p2 = &b;

*p1 += 5;          // Step 1
p1 = p2;           // Step 2
*p1 += 5;          // Step 3
```

Step	a	b	p1 →	p2 →	What happened
Init	10	20	&a	&b	Setup
1	15	20	&a	&b	*p1 is a, so a += 5
2	15	20	&b	&b	p1 now points to b (a is unchanged)
3	15	25	&b	&b	*p1 is now b, so b += 5

Key insight: Step 2 changes *where p1 points*, NOT the value it points to. After step 2, a still exists at its original address — we just don't have a pointer to it anymore.

6. Arrays and Strings

Arrays

An array is a **contiguous block of same-type elements** in memory:

```
uint16_t readings[5] = {100, 200, 300, 400, 500};
```

Index:	[0]	[1]	[2]	[3]	[4]
Value:	100	200	300	400	500
Address:	0x100	0x102	0x104	0x106	0x108

(each uint16_t is 2 bytes apart)

```
readings[0]    // 100 (first element)
readings[4]    // 500 (last element)
readings[5]    // BUG: out of bounds! Reads garbage / causes crash
```

No Bounds Checking

C does **not** check array bounds. Accessing `readings[5]` in a 5-element array reads whatever is next in memory — no error, no warning, just wrong data or a crash.

Always verify `index < size` before accessing an array.

Initializing arrays

```
uint8_t buf[10] = {0};          // All 10 elements set to 0
uint8_t buf[10] = {1, 2, 3};    // First 3 set, rest are 0
uint8_t buf[10];                // UNINITIALIZED - contains random RAM values!
```

```
// Using memset (include <string.h>)
uint8_t buf[256];
memset(buf, 0, sizeof(buf)); // Set all bytes to 0
memset(buf, 0xFF, sizeof(buf)); // Set all bytes to 0xFF
```

Strings in C

A string in C is an array of **char** terminated by a **null character** `'\0'` (value 0):

```
char msg[] = "Hello";
// Compiler actually creates: {'H', 'e', 'l', 'l', 'o', '\0'}
// sizeof(msg) = 6 (5 chars + null terminator)
// strlen(msg) = 5 (does NOT count the null terminator)
```

```
Index:  [0]  [1]  [2]  [3]  [4]  [5]
Value:  'H'  'e'  'l'  'l'  'o'  '\0'
Hex:    0x48 0x65 0x6C 0x6C 0x6F 0x00
```

String functions (from <string.h>)

```
#include <string.h>

strlen(s)           // Length (not counting '\0')
strcmp(a, b)        // Compare: 0 if equal, <0 or >0 otherwise
strncpy(dst, src, n) // Copy at most n chars (safer than strcpy)
memcpy(dst, src, n)  // Copy n bytes (works for any data, not just strings)
memset(buf, val, n)  // Fill n bytes with val
```

Never Use strcpy

`strcpy(dst, src)` copies until it finds `'0'` — if `src` is longer than `dst`, it overflows the buffer. Always use `strncpy(dst, src, sizeof(dst) - 1)` and manually null-terminate.

printf format specifiers (for embedded debugging)

```
uint32_t x = 42;
printf("%u\r\n", x); // unsigned decimal: "42"
printf("%d\r\n", (int)x); // signed decimal: "42"
printf("0x%08X\r\n", x); // hex, 8 digits, zero-padded: "0x0000002A"
printf("%lu\r\n", x); // %lu for uint32_t on ARM (long unsigned)

char *name = "STM32";
printf("MCU: %s\r\n", name); // string: "MCU: STM32"

// Always use \r\n (carriage return + newline) for serial terminals
```

7. Bit Manipulation — The Complete Guide

Bit manipulation is **the most important C skill for embedded work**. Every peripheral register is controlled by setting, clearing, and checking individual bits.

Binary review

An 8-bit register has bits numbered 7 (MSB) down to 0 (LSB):

Bit position:	7	6	5	4	3	2	1	0
Bit value:	128	64	32	16	8	4	2	1
Example:	0	0	1	0	1	0	1	0

= 0x2A = 42

Bitwise operators

Operator	Name	Rule	Example (8-bit)
&	AND	1 only if both bits are 1	0b1100 & 0b1010 = 0b1000
\	OR	1 if either bit is 1	0b1100 \ 0b1010 = 0b1110
^	XOR	1 if bits are different	0b1100 ^ 0b1010 = 0b0110
~	NOT	Flips every bit	~0b1100 = 0b...0011
<<	Left shift	Move bits left, fill with 0	1 << 5 = 0b00100000
>>	Right shift	Move bits right	0b11000000 >> 6 = 0b11

The four register operations

Set a bit (turn ON, make 1)

Use OR with a mask that has a 1 at the target position:

```
reg |= (1 << 5);    // Set bit 5, leave all others unchanged
```

reg:	1001 0010	(before)
mask:	0010 0000	(1 << 5)
	-----	OR
result:	1011 0010	(bit 5 is now 1, others unchanged)

Clear a bit (turn OFF, make 0)

Use AND with a mask that has a 0 at the target position (invert the mask with ~):

```
reg &= ~(1 << 5);    // Clear bit 5, leave all others unchanged
```

mask:	0010 0000	(1 << 5)
~mask:	1101 1111	(inverted)
reg:	1011 0010	(before)
	-----	AND
result:	1001 0010	(bit 5 is now 0, others unchanged)

Toggle a bit (flip)

Use XOR with a mask:

```
reg ^= (1 << 5);    // Flip bit 5
```

Check a bit (read)

Use AND to isolate the bit, then test if non-zero:

```
if (reg & (1 << 5)) {  
    // Bit 5 is set (1)  
} else {  
    // Bit 5 is clear (0)  
}
```

Multi-bit fields

Some register fields span multiple bits. To write a 2-bit field at bits [3:2]:

```
// Clear the field first, then set new value  
reg &= ~(0x3 << 2);    // Clear bits 3:2 (0x3 = 0b11, two-bit mask)  
reg |= (0x2 << 2);    // Set bits 3:2 to 0b10
```

General pattern for an N-bit field starting at position P:

```
reg &= ~(((1 << N) - 1) << P);    // Clear N bits starting at position P  
reg |= (value << P);              // Write new value into the field
```

Worked example: Configure GPIO mode

The STM32 MODER register uses 2 bits per pin. Pin 5 occupies bits [11:10]:

MODER[11:10]	Mode
00	Input
01	Output
10	Alternate function
11	Analog

```
// Set PA5 to output mode (01):  
GPIOA->MODER &= ~(0x3 << 10);    // Clear bits 11:10  
GPIOA->MODER |= (0x1 << 10);    // Set bits 11:10 to 01 (output)  
  
// Set PA5 to alternate function mode (10):  
GPIOA->MODER &= ~(0x3 << 10);    // Clear bits 11:10  
GPIOA->MODER |= (0x2 << 10);    // Set bits 11:10 to 10 (AF)
```

Why clear first? If the field already contains 11 and you OR in 01, you get 11 (OR never clears bits). You must clear the field to zero, *then* set the desired value.

Bit Manipulation Cheat Sheet

<code>reg = (1 << n);</code>	Set bit n
<code>reg &= ~(1 << n);</code>	Clear bit n
<code>reg ^= (1 << n);</code>	Toggle bit n
<code>if (reg & (1 << n))</code>	Test bit n
<code>reg &= ~(0x3 << P); reg = (val << P);</code>	Write 2-bit field at position P

8. Structs

What is a struct?

A struct groups related variables into a single unit:

```
typedef struct {
    uint32_t timestamp;
    uint16_t adc_value;
    uint8_t  channel;
    uint8_t  flags;
} measurement_t;

measurement_t m;           // Declare a variable of this type
m.timestamp = 12345;       // Access members with dot (.)
m.adc_value = 2048;
m.channel   = 3;
m.flags     = 0x01;
```

Struct access: dot vs arrow

```
measurement_t m;           // m is a struct
measurement_t *p = &m;     // p is a pointer to a struct

m.timestamp = 100;         // Dot: used with struct variables
p->timestamp = 100;         // Arrow: used with struct POINTERS
(*p).timestamp = 100;      // Same as arrow (but uglier)
```

Rule: Use `.` for struct variables, `->` for struct pointers. In embedded code, you almost always work with pointers (HAL handles), so `->` is far more common.

Struct memory layout and padding

The compiler inserts invisible **padding bytes** to align members to their natural boundaries:

```
struct Bad {
    uint8_t a;           // offset 0 (1 byte)
                          // offset 1-3: 3 bytes PADDING
    uint32_t b;          // offset 4 (4 bytes)
    uint8_t c;           // offset 8 (1 byte)
                          // offset 9-11: 3 bytes PADDING
};
// sizeof(struct Bad) = 12
```

```

struct Good {
    uint32_t b;    // offset 0  (4 bytes)
    uint8_t  a;    // offset 4  (1 byte)
    uint8_t  c;    // offset 5  (1 byte)
                    // offset 6-7: 2 bytes PADDING
};
// sizeof(struct Good) = 8

```

Rule: Order struct members from **largest to smallest** to minimize padding.

Why structs matter in embedded

Every STM32 peripheral register block is defined as a struct:

```

// From the CMSIS header (stm32f401xe.h):
typedef struct {
    __IO uint32_t MODER;    // offset 0x00
    __IO uint32_t OTYPER;   // offset 0x04
    __IO uint32_t OSPEEDR;  // offset 0x08
    __IO uint32_t PUPDR;    // offset 0x0C
    __IO uint32_t IDR;       // offset 0x10
    __IO uint32_t ODR;       // offset 0x14
    __IO uint32_t BSRR;      // offset 0x18
    __IO uint32_t LCKR;      // offset 0x1C
    __IO uint32_t AFR[2];    // offset 0x20
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) 0x40020000)

```

When you write `GPIOA->ODR`, the compiler calculates $0x40020000 + 0x14 = 0x40020014$ and reads/writes that address. **The struct maps directly to hardware.**

9. The volatile Keyword

The problem

```

uint32_t tick = 0;    // NOT volatile

void TIM2_IRQHandler(void) {
    tick++;            // ISR increments tick
}

int main(void) {
    HAL_TIM_Base_Start_IT(&htim2);
    while (tick < 1000) { }    // Wait for 1000 ticks
    printf("Done!\r\n");
}

```

With optimization (`-O2`), the compiler sees that `main()` never writes to `tick`, so it loads `tick` into a CPU register once and loops forever on that cached value. The ISR updates `tick` in RAM, but `main` never re-reads it. **The program hangs.**

The solution

```
volatile uint32_t tick = 0;    // volatile forces re-read from RAM

int main(void) {
    HAL_TIM_Base_Start_IT(&htim2);
    while (tick < 1000) { }    // Compiler reads tick from RAM every iteration
    printf("Done!\r\n");        // Works correctly now
}
```

When to use volatile

Scenario	volatile?	Reason
Hardware register (GPIOA->IDR)	Yes	Hardware changes the value
Variable written by ISR, read by main	Yes	ISR modifies it outside normal flow
Variable shared between RTOS threads	Yes	Other thread modifies it
Local variable in a function	No	Only this code touches it
Constant configuration value	No	Never changes

volatile in One Sentence

volatile tells the compiler: “don’t cache this — someone else can change it behind your back.” Always use it for hardware registers and ISR-shared variables.

10. The Preprocessor

The preprocessor runs **before** the compiler. It does text substitution.

#define — Constants and macros

```
#define LED_PIN      5
#define LED_PORT     GPIOA
#define LED_ON()     (LED_PORT->BSRR = (1 << LED_PIN))
#define LED_OFF()    (LED_PORT->BSRR = (1 << (LED_PIN + 16)))

// After preprocessing, LED_ON() becomes:
// (GPIOA->BSRR = (1 << 5))
```

Always parenthesize macro arguments:

```
#define SQUARE(x)    ((x) * (x))    // GOOD: parenthesized

#define BAD_SQ(x)    x * x          // BAD
// BAD_SQ(2+3) expands to 2+3 * 2+3 = 2+6+3 = 11, not 25!
```

#include — File inclusion

```
#include <stdint.h>          // System header: compiler search path
#include "stm32f4xx_hal.h"    // Project header: current directory first
```

`#include` literally pastes the file contents at that point. Header files contain declarations so multiple `.c` files can share function prototypes and type definitions.

Include guards

Prevent a header from being included twice (which causes “duplicate definition” errors):

```
// my_header.h
#ifndef MY_HEADER_H          // If not defined...
#define MY_HEADER_H          // ...define it (first time only)

// Declarations go here
void my_function(void);

#endif // MY_HEADER_H
```

#ifdef / #ifndef — Conditional compilation

```
#define DEBUG_UART

#ifdef DEBUG_UART
    printf("tick = %lu\r\n", tick);    // Only compiled if DEBUG_UART is defined
#endif
```

Used heavily in embedded to enable/disable debug output without changing logic.

11. Common Embedded C Patterns

Pattern 1: Flag-based main loop

```
volatile uint8_t flag_timer = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        flag_timer = 1;          // ISR sets flag
    }
}

int main(void) {
    // ... init code ...
    while (1) {
        if (flag_timer) {
            flag_timer = 0;      // Clear flag
            process_data();      // Do work in main context, not ISR
        }
    }
}
```



```

    }
}
}

```

Why: Keep ISRs short. Do the actual work (logging, computation, communication) in the main loop where you can safely call HAL functions.

Pattern 2: Circular buffer

```

#define BUF_SIZE 64

uint8_t buffer[BUF_SIZE];
volatile uint8_t head = 0;    // Write position (ISR writes here)
volatile uint8_t tail = 0;    // Read position (main reads here)

// ISR: add byte to buffer
void USART2_IRQHandler(void) {
    uint8_t next = (head + 1) % BUF_SIZE;
    if (next != tail) {        // Not full?
        buffer[head] = USART2->DR;
        head = next;
    }
}

// Main: read from buffer
uint8_t available(void) { return (head - tail + BUF_SIZE) % BUF_SIZE; }

```

Pattern 3: Timeout pattern

```

uint32_t start = HAL_GetTick();
while (!(USART2->SR & (1 << 5))) {    // Wait for RXNE
    if (HAL_GetTick() - start > 1000) { // 1 second timeout
        return HAL_TIMEOUT;
    }
}

```

Never use while(1) to wait for hardware without a timeout. If the hardware doesn't respond, your firmware hangs forever.

Pattern 4: Lookup tables (avoid floating-point math)

```

// Instead of: sin(angle * PI / 180) at runtime (slow, needs FPU)
// Precompute a table:
const uint16_t sine_table[91] = {
    0, 175, 349, 523, 698, 872, 1045, ... // sin(x) * 10000
};

uint16_t sin_lookup(uint8_t degrees) {
    if (degrees > 90) return sine_table[180 - degrees];
    return sine_table[degrees];
}

```

12. Practice Problems

Work through these with a compiler. Predict the output first, then verify.

Problem 1: Pointer tracing

```
int a = 5, b = 10, c = 15;
int *p = &a;
int *q = &b;

*p = *q;          // Line 1
p = &c;            // Line 2
*p = *q + *p;     // Line 3

printf("a=%d b=%d c=%d\n", a, b, c);
```

Answer

Line 1: *p is a, *q is b. So a = 10.
Line 2: p now points to c. Values unchanged.
Line 3: *p is c (15), *q is b (10). c = 10 + 15 = 25.
Output: a=10 b=10 c=25

Problem 2: Bit manipulation

```
uint8_t reg = 0xA5;    // Binary: 1010 0101

reg |= (1 << 1);        // Line 1
reg &= ~(1 << 5);       // Line 2
reg ^= (1 << 7);        // Line 3

printf("0x%02X\n", reg);
```

Answer

Start: 1010 0101 (0xA5)
Line 1: Set bit 1 → 1010 0111 (0xA7)
Line 2: Clear bit 5 → 1000 0111 (0x87)
Line 3: Toggle bit 7 → 0000 0111 (0x07)
Output: 0x07

Problem 3: Struct and arrow operator

```
typedef struct { int x; int y; } point_t;

point_t a = {3, 4};
point_t *p = &a;

p->x = p->x + p->y;
p->y = p->x - p->y;

printf("x=%d y=%d\n", a.x, a.y);
```

Answer

$p \rightarrow x = 3 + 4 = 7$. Then $p \rightarrow y = 7 - 4 = 3$.

Output: x=7 y=3

Problem 4: Array and sizeof

```
uint32_t data[8] = {0};
uint32_t *p = data;

printf("A: %lu\n", sizeof(data));    // ?
printf("B: %lu\n", sizeof(p));       // ?
printf("C: %lu\n", sizeof(*p));      // ?
printf("D: %lu\n", sizeof(data) / sizeof(data[0])); // ?
```

Answer

A: 32 (8 elements $\times$ 4 bytes each)

B: 4 (pointer size on 32-bit ARM)

C: 4 (*p is uint32_t, which is 4 bytes)

D: 8 (total size $\div$ element size = number of elements)

Pattern D is a common idiom for getting array length at the declaration site.

Problem 5: Multi-bit register field

You need to set the MODER register for PA7 (bits [15:14]) to alternate function mode (10), without disturbing any other bits. The register currently contains 0x00000280.

Answer

`GPIOA->MODER &= ~(0x3 << 14);` clears bits 15:14.

`GPIOA->MODER |= (0x2 << 14);` sets bits 15:14 to 10.

Before: 0x00000280 = ...0000 0010 1000 0000

After clear: bits 15:14 $\rightarrow$ 00, rest unchanged = 0x00000280 (they were already 00)

After set: bits 15:14 $\rightarrow$ 10 = 0x00008280

Problem 6: Volatile reasoning

```
uint32_t counter = 0;           // Missing volatile

void SysTick_Handler(void) { counter++; }

int main(void) {
    while (counter < 100) {}     // Waits for 100 SysTick interrupts
    do_something();
}
```

Question: What happens when compiled with `-O2`? How do you fix it?

Answer

The compiler sees that `main()` never writes to `counter`. At `-O2`, it loads `counter` into a CPU register once (value 0) and checks that register forever. The ISR updates `counter` in RAM, but `main` never re-reads RAM.

Result: Infinite loop. `do_something()` never runs.

Fix: Declare as volatile `uint32_t counter = 0;`

Quick Reference Card

Type sizes (ARM Cortex-M4)

Type	Bytes	Type	Bytes
<code>uint8_t</code> / <code>int8_t</code>	1	<code>float</code>	4
<code>uint16_t</code> / <code>int16_t</code>	2	<code>double</code>	8
<code>uint32_t</code> / <code>int32_t</code>	4	pointer (*)	4

Operator precedence (most common, high to low)

Precedence	Operators	Description
1 (highest)	<code>() [] -> .</code>	Grouping, access
2	<code>! ~ * & ++ -- (type)</code>	Unary, dereference, cast
3	<code>* / %</code>	Multiply, divide, modulo
4	<code>+ -</code>	Add, subtract
5	<code><< >></code>	Bit shift
6	<code>< <= > >=</code>	Comparison
7	<code>== !=</code>	Equality
8	<code>&</code>	Bitwise AND
9	<code>^</code>	Bitwise XOR
10	<code>\ </code>	Bitwise OR
11	<code>&&</code>	Logical AND
12	<code>\ \ </code>	Logical OR
13 (lowest)	<code>= += -= \ = &=</code>	Assignment

Precedence Trap

`if (reg & (1 << 5) == 0)` does NOT work! `==` has higher precedence than `&`, so it evaluates as `reg & ((1 << 5) == 0) → reg & 0 → always 0`.

Fix: `if ((reg & (1 << 5)) == 0)` — always parenthesize bitwise operations in conditions.

printf format specifiers (embedded)

Specifier	Type	Example output
<code>%u</code>	unsigned int	42
<code>%lu</code>	<code>uint32_t</code>	4294967295
<code>%d</code>	int	-42
<code>%x / %X</code>	hex (lower/upper)	2a / 2A
<code>%08X</code>	hex, 8 digits, zero-padded	0000002A
<code>%s</code>	string (<code>char *</code>)	Hello
<code>%c</code>	single char	A

Specifier	Type	Example output
%p	pointer	0x20000100
%%	literal %	%